LAB: EXTI & SysTick

Date: 2023-09-29

Author/Partner: HWANG Moo Yeol

Github: repository link

Demo Video: Problem1 Demo video, Problem2 Demo video

PDF version: LAB_EXTI_&_SysTick PDF

Introduction

In this lab, you are required to create two simple programs using interrupt:

(1) displaying the number counting from 0 to 9 with Button Press

(2) counting at a rate of 1 second

You must submit

- LAB Report (*.md & *.pdf)
- Zip source files(main*.c, ecRCC2.h, ecGPIO2.h, ecSysTick2.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - 4 LEDs and load resistance
    - 7-segment display(5101ASR)
    - Array resistor (330 ohm)
    - breadboard

Software

- Keil uVision, CMSIS, EC_HAL library

Problem 1: Counting numbers on 7-Segment using EXTI Button

1. Declare and define the following functions in your library : **ecEXTI2.h**, **ecEXTI.h**

```c
void EXTI_init(PinName_t pinName, int trig_type, int priority);
void EXTI_enable(uint32_t pin);  // mask in IMR
void EXTI_disable(uint32_t pin);  // unmask in IMR
uint32_t  is_pending_EXTI(uint32_t pin);
void clear_pending_EXTI(uint32_t pin);
```

```c
#include "ecGPIO2_student.h"
#include "ecSysTick2.h"
#include "ecEXTI2.h"


void EXTI_init(PinName_t pinName, int trig_type,int priority){
    //Port, Pin Configuration
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);
    // SYSCFG peripheral clock enable
    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;

    // Connect External Line to the GPIO
    int EXTICR_port;
    if          (Port == GPIOA) EXTICR_port = 0;
    else if (Port == GPIOB) EXTICR_port = 1;
    else if (Port == GPIOC) EXTICR_port = 2;
    else if (Port == GPIOD) EXTICR_port = 3;
    else                                        EXTICR_port = 4;

    SYSCFG->EXTICR[pin/4] &= ~(0b1111<<((pin%4)*4));            // clear 4 bits
    SYSCFG->EXTICR[pin/4] |= EXTICR_port<<((pin%4)*4);          // set 4 bits

    // Configure Trigger edge
    if (trig_type == FALL) EXTI->FTSR |= 1 << pin;   // Falling trigger enable
    else if (trig_type == RISE) EXTI->RTSR |= 1 << pin ;   // Rising trigger
enable
    else if (trig_type == BOTH) {           // Both falling/rising trigger
enable
        EXTI->RTSR |= 1 << pin;
        EXTI->FTSR |= 1 << pin;
    }

    // Configure Interrupt Mask (Interrupt enabled)
    EXTI->IMR  |= 1 << pin ;     // not masked


    // NVIC(IRQ) Setting
    int EXTI_IRQn = 0;

    if (pin < 5)    EXTI_IRQn = EXTI0_IRQn + pin;
    else if (pin < 10)  EXTI_IRQn = EXTI9_5_IRQn;
    else            EXTI_IRQn = EXTI15_10_IRQn;

    NVIC_SetPriority(EXTI_IRQn, priority);  // EXTI priority
    NVIC_EnableIRQ(EXTI_IRQn);  // EXTI IRQ enable
}
```

```c
void EXTI_enable(PinName_t pinName) {
    //Port, Pin Configuration
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    EXTI->IMR |= 1<< pin;     // not masked (i.e., Interrupt enabled)
}

void EXTI_disable(PinName_t pinName) {
    //Port, Pin Configuration
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    EXTI->IMR &= ~(1<< pin);     // masked (i.e., Interrupt disabled)
}

uint32_t is_pending_EXTI(PinName_t pinName) {
    //Port, Pin Configuration
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    uint32_t EXTI_PRx = 1 << pin ;      // check  EXTI pending
    return ((EXTI->PR & EXTI_PRx) == EXTI_PRx);
}


void clear_pending_EXTI(PinName_t pinName) {
    //Port, Pin Configuration
    GPIO_TypeDef *Port;
    unsigned int pin;
    ecPinmap(pinName,&Port,&pin);

    EXTI->PR |= 1 << pin;     // clear EXTI pending
}
```

Procedure

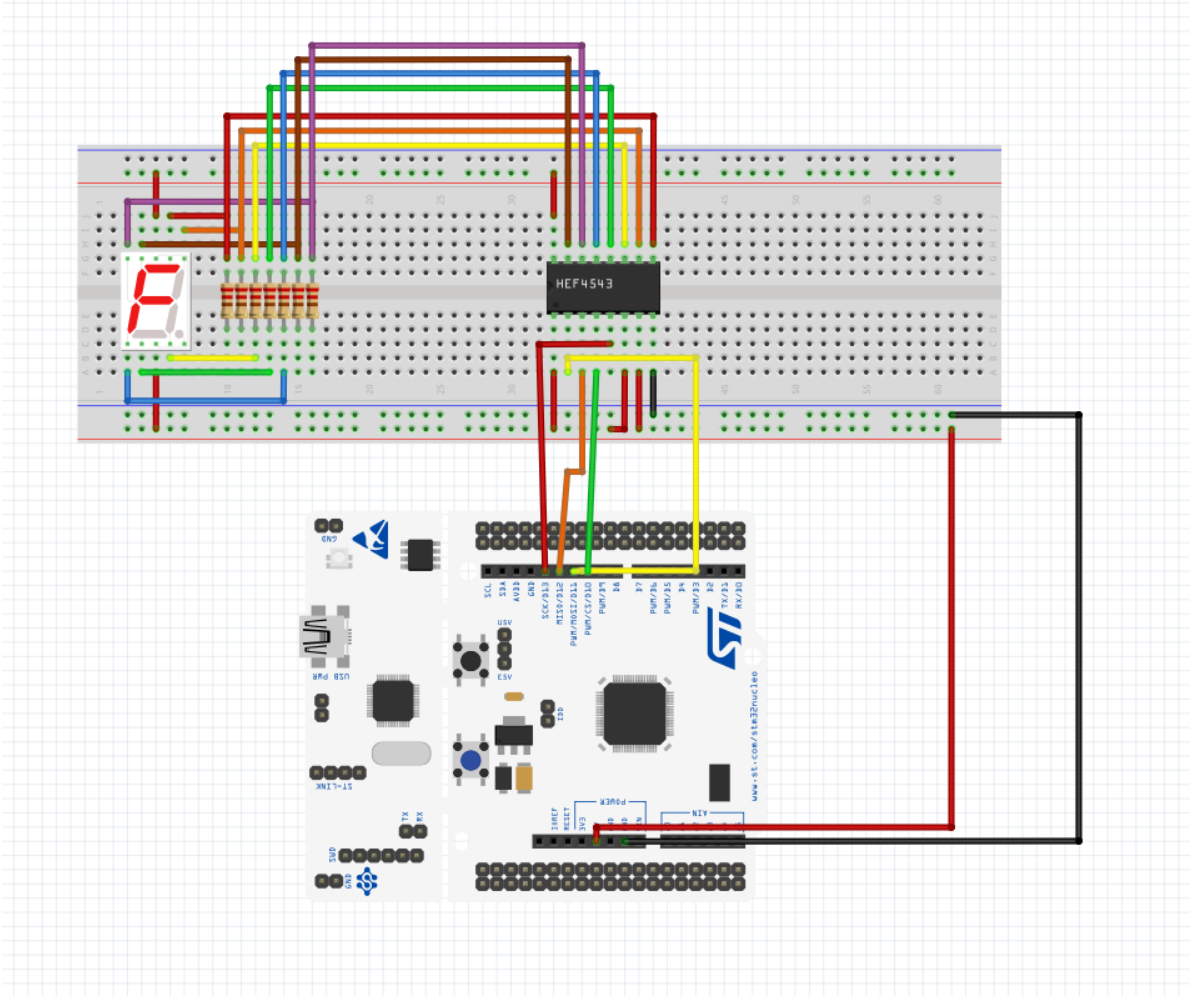
1. Create a code to display the number counting from 0 to 9 and repeating.
    - Count up only by pressing the push button (External Interrupt)

Configuration

Digital In for Button (B1)	Digital Out for 7-Segment decoder
Digital In	Digital Out
PC13	PA7, PB6, PC7, PA9
PULL-UP	Push-Pull, No PullUp-PullDown, Medium Speed

Circuit Diagram

> Circuit diagram for 7-segment using EXTI button.



Discussion

1. We can use two different methods to detect an external signal: polling and interrupt. What are the advantages and disadvantages of each approach?

> -Polling method is a method of continuously reading hardware changes, periodically inspecting whether an event is performed, and executing an event when it receives a corresponding signal.
>
> advantages : easy to implement, may be useful for small programs
>
> disadvantages: less inefficient, have a high system load
>
> -Interrupt method is which the CPU detects changes in hardware and recognizes inputs from the outside.
>
> advantages : high inefficient, have a low system load
>
> disadvantages: hard to implement, Time difference between tasks

2. What would happen if the EXTI interrupt handler does not clear the interrupt pending flag? Check with your code

> If the interrupt handler does not clear the interrupt pending flag after processing the interrupt, the MCU would mistakenly think another interrupt request has arrived and then repeatedly execute the interrupt handler.

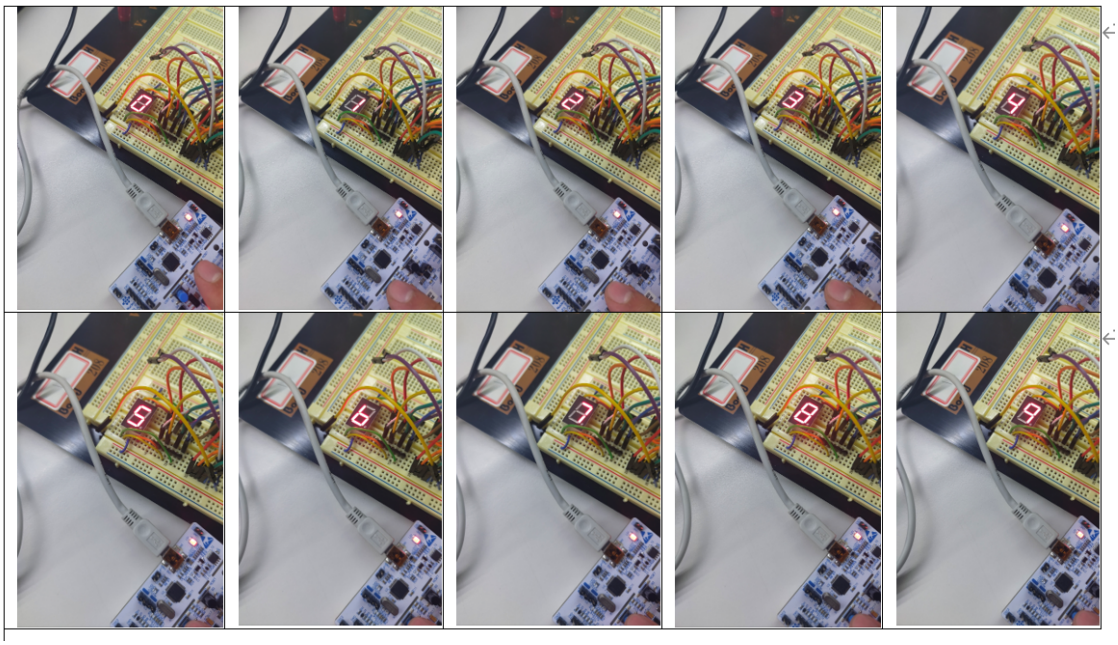
> When check with my code the 7-segment did not change and stop at number '8' . I guess the interrupt handler have been implemented very quickly.

Code

```
#include "stm32f4xx.h"
#include "ecGPIO2_student.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"


//#define LED_PIN   PA_5
#define BUTTON_PIN PC_13

// Initialiization
void setup(void);

// count for 7-segment
unsigned int cnt = 0;

int main(void) {
    setup();
    while (1) {}
}

//EXTI for Pin 13
void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {

        sevensegment_display(cnt % 10);     // sevensegment_display from
previous lab
        cnt++;
        if (cnt > 9) cnt = 0;

        clear_pending_EXTI(BUTTON_PIN);   // clear pending
    }
}


// Initialiization by configuration
void setup(void) {
    RCC_PLL_init();                        // system cloclk PLL
    SysTick_init();                        // SysTick_enable

    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    sevensegment_display_init(PA_7, PB_6, PC_7, PA_9);  // Decoder input A,B,C,D
    GPIO_otype(PA_7, PUSH_PULL);
    GPIO_otype(PB_6, PUSH_PULL);
    GPIO_otype(PC_7, PUSH_PULL);
    GPIO_otype(PA_9, PUSH_PULL);

    GPIO_pupd(PA_7, NO_PUPD);
```

```
    GPIO_pupd(PB_6, NO_PUPD);
    GPIO_pupd(PC_7, NO_PUPD);
    GPIO_pupd(PA_9, NO_PUPD);

    GPIO_ospeed(PA_7, MEDIUM_SPEED);
    GPIO_ospeed(PB_6, MEDIUM_SPEED);
    GPIO_ospeed(PC_7, MEDIUM_SPEED);
    GPIO_ospeed(PA_9, MEDIUM_SPEED);

    EXTI_init(BUTTON_PIN, FALL, 0);  // Priority Highest(0) External Interrupt
}
```

Results



[demo video link]

Problem 2: Counting numbers on 7-Segment using SysTick

Display the number 0 to 9 on the 7-segment LED at the rate of 1 sec. After displaying up to 9, then it should display '0' and continue counting.

When the button is pressed, the number should be reset '0' and start counting again.

1. Declare and define the following functions in your library : **ecSysTick2.h**, **ecSysTick.h**

```
void SysTick_init(uint32_t msec);
void delay_ms(uint32_t msec);
uint32_t SysTick_val(void);
void SysTick_reset (void);
void SysTick_enable(void);
void SysTick_disable (void)
```

```c
#include "ecSysTick2.h"



#define MCU_CLK_PLL 84000000
#define MCU_CLK_HSI 16000000

volatile uint32_t msTicks;

//EC_SYSTEM_CLK

void SysTick_init(void){
    //  SysTick Control and Status Register
    SysTick->CTRL = 0;                                          // Disable
SysTick IRQ and SysTick Counter

    // Select processor clock
    // 1 = processor clock;  0 = external clock
    SysTick->CTRL |= SysTick_CTRL_CLKSOURCE_Msk;

    // uint32_t MCU_CLK=EC_SYSTEM_CLK
    // SysTick Reload Value Register
    SysTick->LOAD = MCU_CLK_PLL / 1000 - 1;                     // 1ms, for HSI
PLL = 84MHz.

    // SysTick Current Value Register
    SysTick->VAL = 0;

    // Enables SysTick exception request
    // 1 = counting down to zero asserts the SysTick exception request
    SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;

    // Enable SysTick IRQ and SysTick Timer
    SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;

    NVIC_SetPriority(SysTick_IRQn, 16);     // Set Priority to 1
    NVIC_EnableIRQ(SysTick_IRQn);           // Enable interrupt in NVIC
}



void SysTick_Handler(void){
    SysTick_counter();
}

void SysTick_counter(){
    msTicks++;
}


void delay_ms (uint32_t mesc) {
  uint32_t curTicks;

  curTicks = msTicks;
  while ((msTicks - curTicks) < mesc);

  msTicks = 0;
```

```
}

void SysTick_reset(void) {
    SysTick->VAL = 0;                              // SysTick Current Value
Register
}

uint32_t SysTick_val(void) {
    return SysTick->VAL;
}

void SysTick_enable(void){
    SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;
}

void SysTick_disable (void) {
    SysTick->CTRL = 0;
}
```

Procedure

1. Create a code to display the number counting from 0 to 9 and repeats at the rate of 1 second.
2. When the button is pressed, it should start from '0' again.

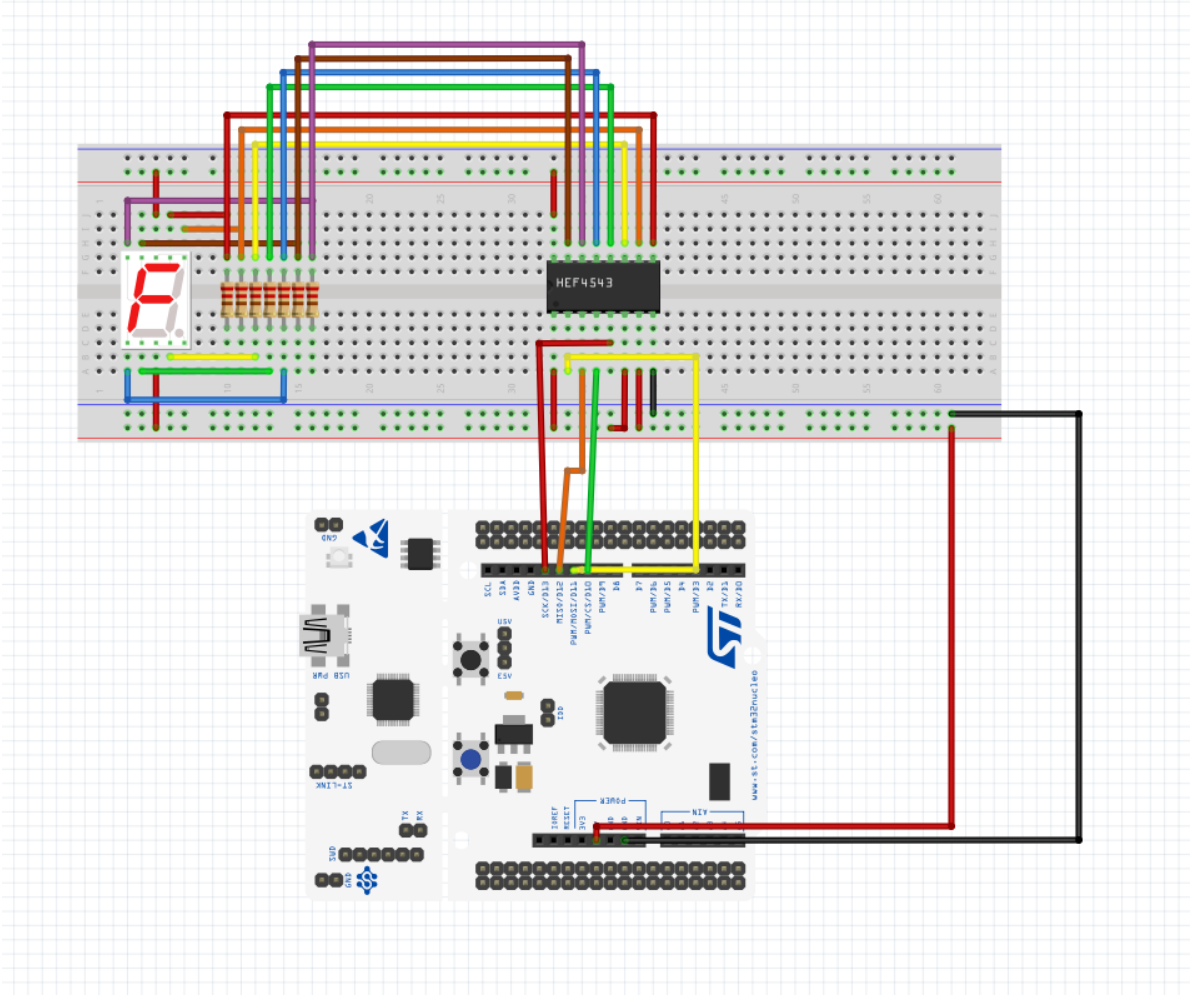
> Use EXTI for this button reset.

Configuration

Digital In for Button (B1)	Digital Out for 7-Segment decoder
Digital In	Digital Out
PC13	PA7, PB6, PC7, PA9
PULL-UP	Push-Pull, No Pull-up-Pull-down, Medium Speed

Circuit Diagram

> Circuit diagram for 7-segment using systick

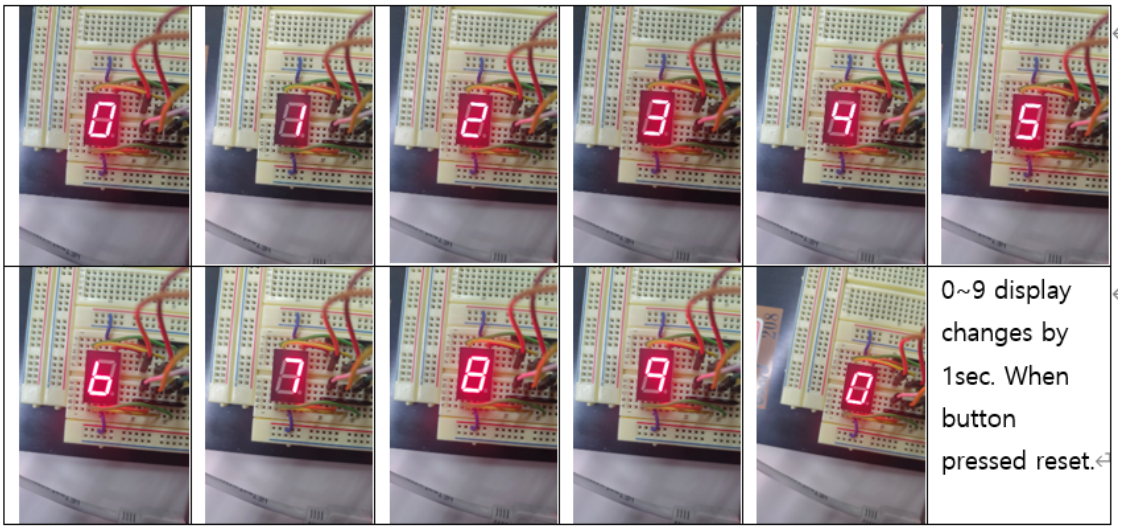


Code

```
#include "stm32f4xx.h"
#include "ecGPIO2_student.h"
#include "ecRCC2.h"
#include "ecEXTI2.h"
#include "ecSysTick2.h"


#define BUTTON_PIN PC_13


// Initialiization
void setup(void);
void SysTick_Handler(void);

// count for 7-segment
unsigned int cnt = 0;


int main(void) {
    setup();
    while (1) {
    sevensegment_display(cnt % 10);
    cnt++;
    if (cnt > 9) cnt = 0;
    delay_ms(1000);        //  counting by systick_handler() msticks
```

```c
    }
}


//EXTI for Pin 13
void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(BUTTON_PIN)) {

        cnt = 0;                          //EXTI for reseting timer to '0'

        clear_pending_EXTI(BUTTON_PIN);
    }
}

// Initialiization
void setup(void) {
    RCC_PLL_init();                 // system cloclk PLL
    SysTick_init();                // calls SysTick_Handler()

    GPIO_init(BUTTON_PIN, INPUT);  // calls RCC_GPIOC_enable()
    GPIO_pupd(BUTTON_PIN, PULL_UP);

    sevensegment_display_init(PA_7, PB_6, PC_7, PA_9);  // Decoder input A,B,C,D
    GPIO_otype(PA_7, PUSH_PULL);
    GPIO_otype(PB_6, PUSH_PULL);
    GPIO_otype(PC_7, PUSH_PULL);
    GPIO_otype(PA_9, PUSH_PULL);

    GPIO_pupd(PA_7, NO_PUPD);
    GPIO_pupd(PB_6, NO_PUPD);
    GPIO_pupd(PC_7, NO_PUPD);
    GPIO_pupd(PA_9, NO_PUPD);

    GPIO_ospeed(PA_7, MEDIUM_SPEED);
    GPIO_ospeed(PB_6, MEDIUM_SPEED);
    GPIO_ospeed(PC_7, MEDIUM_SPEED);
    GPIO_ospeed(PA_9, MEDIUM_SPEED);
    EXTI_init(BUTTON_PIN, FALL, 0);   // Priority Highest(0) External Interrupt
}
```

Results



demo video link

Reference

> 1. Yifeng Zhu - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (2018)
> 2. polling과 interrupt의 차이점

Troubleshooting

[-]